

图像处理：第六次作业

Illusionna 2025.....004 人工智能学院

02:44, Monday 12th January, 2026

目录

1	灰度值及其在图像中出现的概率	1
(a)	哈夫曼编码构建最优二叉树	1
(b)	哈夫曼编码节省的存储空间	2
(c)	哈夫曼编码在图像压缩中的优劣性	3
2	算术解码	3
3	名词解释	4
(a)	信源编码与解码器	4
(b)	信道编码与解码器	4
(c)	信息量 / 自信息	5
(d)	熵	5
(e)	条件熵	5
(f)	互信息	5
(g)	信道容量	5
4	海明码的编码解码计算方法	6
(a)	编码方法	6
(b)	解码与纠错	6
(c)	矩阵表示法	7
5	证明二元表达式	7

1 灰度值及其在图像中出现的概率

(a) 哈夫曼编码构建最优二叉树

表 1: 一组灰度值及其在图像中出现的频率

灰度值	频率
$0'$	45
$1'$	35
$2'$	20
$3'$	15
$4'$	15
$5'$	10
$6'$	5
$7'$	5

Step 1: 按照频率从小到大排序

表 2: 一组灰度值及其在图像中出现的频率

灰度值	频率
$6'$	5
$7'$	5
$5'$	10
$3'$	15
$4'$	15
$2'$	20
$1'$	35
$0'$	45

Step 2: 依次取最小频率的两个灰度节点，其权重为两者之和，构建哈夫曼树

- $6' + 7' = N1 = 5 + 5 = 10$
- $5' + N1 = N2 = 10 + 10 = 20$
- $3' + 4' = N3 = 15 + 15 = 30$
- $2' + N2 = N4 = 20 + 20 = 40$
- $N3 + 1' = N5 = 30 + 35 = 65$
- $N4 + 0' = N6 = 40 + 45 = 85$
- $N5 + N6 = N7 = 65 + 85 = 150$

Step 3: 约定左分支子树编码为 0，右方为 1，构建哈夫曼树

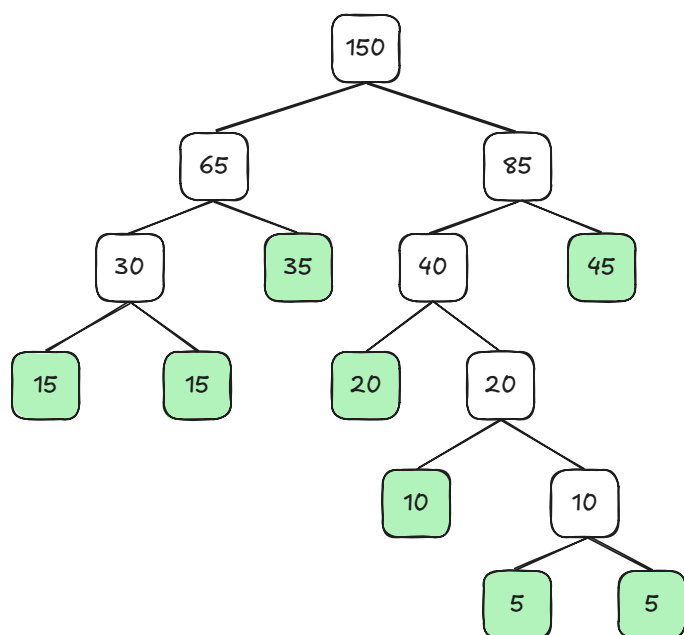


图 1: 最优哈夫曼二叉树

Step 4: 构建哈夫曼编码表

表 3: 一组灰度值及其在图像中出现的频率

灰度值	频率	哈夫曼编码	码长
0'	45	11	2
1'	35	01	2
2'	20	100	3
3'	15	000	3
4'	15	001	3
5'	10	1010	4
6'	5	10110	5
7'	5	10111	5

Step 5: 计算平均码长

$$\bar{L} = \frac{\sum_{i=1}^n f_i \times L_i}{\sum_{i=1}^n f_i} = \frac{45 \times 2 + 35 \times 2 + 20 \times 3 + \dots + 5 \times 5 + 5 \times 5}{45 + 35 + 20 + 15 + 15 + 10 + 5 + 5} = \frac{400}{150} \approx 2.667$$

(b) 哈夫曼编码节省的存储空间

因为总像素（即总频率）为 150，由于每个像素 3 个位数，所以一共 $150 \times 3 = 450$ ，而哈夫曼编码压缩后总大小为 400，所以节省了 50 比特，因此节省比例是 $50 / 450 \approx 0.111$ ，即使用哈夫曼编码可以节省 11.11% 的存储空间。

(c) 哈夫曼编码在图像压缩中的优劣性

哈夫曼编码在数字图像压缩中通常扮演最后一步打包工作，而不是唯一的压缩手段，具体的优劣势如下：

- 优势 1：无损压缩，哈夫曼编码完全基于统计概率，解码后的数据与编码前的完全一致，没有丢失任何信息；
- 优势 2：实现简单且高效，算法逻辑清晰，使用查表法，计算复杂度很低，编码解码速度非常快，适合实时处理；
- 优势 3：符号级最优，在不考虑符号间相关性且必须使用整数位来表示一个符号的前提下，哈夫曼编码被证明是效率最高的编码方式；

- 劣势 1：忽略了像素间的相关性，哈夫曼编码通常把像素当作独立的事件来看，但实际上，一个蓝色像素点附近的像素也大概率是蓝色，如果直接对原始图像做哈夫曼编码，效率并不高，没有利用这种邻居相似的规律，现代标准，如 JPEG 通常先通过 DCT 变化去除相关性，剩下不相关度数据系数再交给哈夫曼编码；
- 劣势 2：整数位限制，哈夫曼编码强制每个符号码长必须是整数，如果符号出现的概率极高，理论上不需要很多位就能表示，但哈夫曼最少也得分配 1 个位，导致浪费，现代通常采用算术编码，逼近分数位，因此比哈夫曼压缩率更高，但计算也更复杂；
- 劣势 3：误码敏感性，由于是变长编码且没有固定的分隔符，一旦数据流中有一个比特发生翻转，比如 0 变为 1，解码器可能会“对齐错误”，导致后面一连串数据全部计算错误。

2 算术解码

对编码模型信息 0.23355 进行解码：

表 4: 算术编码与解码

符号	频率
a	0.2
e	0.3
i	0.1
o	0.2
u	0.1
!	0.1

Step 1: 构建累积概率区间

- a: $0.2 \rightarrow [0, 0.2)$
- e: $0.3 \rightarrow [0.2, 0.5)$
- i: $0.1 \rightarrow [0.5, 0.6)$
- o: $0.2 \rightarrow [0.6, 0.8)$

- u: $0.1 \rightarrow [0.8, 0.9)$
- !: $0.1 \rightarrow [0.9, 1)$

表 5: 算术编码与解码累计区间

符号	频率	累计区间
a	0.2	[0, 0.2)
e	0.3	[0.2, 0.5)
i	0.1	[0.5, 0.6)
o	0.2	[0.6, 0.8)
u	0.1	[0.8, 0.9)
!	0.1	[0.9, 1)

Step 2: 解码迭代过程计算

$$V' = \frac{V - \min}{\max - \min}$$

- 输入: 0.23355, 输出: e, 迭代: $\frac{0.23355 - 0.2}{0.5 - 0.2} \approx 0.11183$
- 输入: 0.11183, 输出: a, 迭代: $\frac{0.11183 - 0}{0.2 - 0} \approx 0.55915$
- 输入: 0.55915, 输出: i, 迭代: $\frac{0.55915 - 0.5}{0.6 - 0.5} \approx 0.59167$
- 输入: 0.59167, 输出: i, 迭代: $\frac{0.59167 - 0.5}{0.6 - 0.5} \approx 0.91667$
- 输入: 0.91667, 输出: ! 为终止程序, 退出, 返回结果 eaii!

3 名词解释

(a) 信源编码与解码器

定义: 信源编码的主要目的是为了提高通信的有效性。它通过压缩数据、去除信源中的冗余信息, 将信源符号转换为适合传输或存储的数字信号, 通常是二进制流。常见的信源编码算法包括哈夫曼编码和算术编码。解码器则是编码的逆过程, 试图无失真或在允许失真范围内恢复原始信源信息。

(b) 信道编码与解码器

定义: 信道编码的主要目的是为了提高通信的可靠性。针对信道中可能存在的噪声和干扰, 信道编码器在信息序列中有规律地增加一些冗余码元, 如校验位, 以便在接收端进行检

错或纠错。解码器利用这些冗余信息来检测或纠正传输过程中产生的错误，从受损的信号中恢复出原始发送的数据。

(c) 信息量 / 自信息

定义：自信息是衡量一个特定事件发生时所带来的信息量，或者是该事件发生前的不确定性。一个事件发生的概率越小，其发生时所包含的信息量就越大。

数学公式：离散随机变量 X 的某个取值 x 发生的概率为 $p(x)$ ，则其自信息 $I(x)$ 定义：

$$I(x) = -\log_2 p(x)$$

单位通常为比特。

(d) 熵

定义：熵，也称为香农熵，表示信源的平均不确定性，或者是信源输出的平均信息量。它是所有可能事件自信息的数学期望。

数学公式：对于离散随机变量 X ，其概率分布为 $p(x_i)$ ，则熵 $H(X)$ 为：

$$H(X) = \sum_i p(x_i) I(x_i) = - \sum_i p(x_i) \log_2 p(x_i)$$

(e) 条件熵

定义：条件熵 $H(Y|X)$ 表示在已知随机变量 X 的条件下，随机变量 Y 的不确定性。它衡量了在接收到 X 后，关于 Y 仍然存在的平均不确定度，通常用于描述信道中的噪声损失。

数学公式：

$$H(Y|X) = - \sum_x \sum_y p(x, y) \log_2 p(y|x)$$

(f) 互信息

定义：互信息 $I(X; Y)$ 表示两个随机变量之间的相关性度量。在通信中，它表示接收端在收到信号 Y 后，获得的关于发送端 X 的信息量。也就是“发送 X 前的不确定性”减去“收到 Y 后 X 的不确定性”。

数学公式：

$$I(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

(g) 信道容量

定义：信道容量是指信道能够无差错传输信息的最大理论速率。根据香农第二定理，信道编码定理，只要信息传输速率低于信道容量，就存在一种编码方式，使得误差概率可以任意小。

数学公式：信道容量 C 是互信息 $I(X;Y)$ 关于输入概率分布 $p(x)$ 的最大值：

$$C = \max_{p(x)} I(X;Y)$$

4 海明码的编码解码计算方法

海明码 (7, 4) 是一种能够纠正 1 位错误的线性分组码，它通过在 4 位数据位中加入 3 位校验位，构成总长为 7 位的码字。

- 码字总长度： $n = 7$
- 数据位长度： $k = 4$
- 校验位长度： $m = n - k = 3$

(a) 编码方法

位置分配

设原始数据为 $\mathbf{d} = (d_1, d_2, d_3, d_4)$ 。在海明码中，校验位 p_i 被放置在 2 的幂次方位置（即第 1, 2, 4 位），数据位按顺序填充剩余位置。生成的码字 $\mathbf{c}$ 结构如下：

$$\mathbf{c} = (p_1, p_2, d_1, p_3, d_2, d_3, d_4)$$

校验位计算

校验位的计算基于模 2 加法，即异或运算，记为 $\oplus$ ，每个校验位负责检查其对应二进制索引中该位为 1 的所有位置：

$$p_1 = d_1 \oplus d_2 \oplus d_4 \quad (\text{校验第1, 3, 5, 7 位})$$

$$p_2 = d_1 \oplus d_3 \oplus d_4 \quad (\text{校验第2, 3, 6, 7 位})$$

$$p_3 = d_2 \oplus d_3 \oplus d_4 \quad (\text{校验第4, 5, 6, 7 位})$$

(b) 解码与纠错

假设接收端收到的码字为 $\mathbf{r} = (r_1, r_2, r_3, r_4, r_5, r_6, r_7)$ ，为了检测错误，我们需要计算伴随式 $\mathbf{S} = (s_3, s_2, s_1)$ 。

伴随式计算

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7$$

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7$$

$$s_3 = r_4 \oplus r_5 \oplus r_6 \oplus r_7$$

错误定位逻辑

将伴随式向量 $\mathbf{S}$ 视为一个二进制数 $(s_3s_2s_1)_{[2]}$:

- 若 $\mathbf{S} = (0,0,0)$, 表示传输无误。
- 若 $\mathbf{S} \neq 0$, 将该二进制数转换为十进制整数 k , 则说明第 k 位发生了错误。
- 纠错操作: 将接收到的第 k 位数据取反 ($0 \rightarrow 1$ 或 $1 \rightarrow 0$) 即可恢复正确数据。

(c) 矩阵表示法

在计算机实现中, 通常使用生成矩阵 G 和校验矩阵 H 进行运算。海明编码过程可以表示为 $\mathbf{c} = \mathbf{d}G$:

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

使用校验矩阵 H 进行解码:

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

伴随式的计算可通过矩阵乘法获得:

$$\mathbf{S}^\top = H \cdot \mathbf{r}^\top \pmod{2}$$

5 证明二元表达式

证明:

$$(A \bullet B)^c = A^c \circ \widehat{B}$$

展开闭运算的定义, 将 $A \bullet B$ 替换为其定义公式 LHS, 再应用腐蚀的对偶性, 根据 $(X \ominus Y)^c = X^c \oplus \widehat{Y}$ 对整体求补集, 然后应用 $(X \oplus Y)^c = X^c \ominus \widehat{Y}$ 膨胀的对偶性, 最后利用 $X \circ Y = (X \ominus Y) \oplus Y$ 开运算的定义, 推导出结果 RHS:

$$\begin{aligned} (A \bullet B)^c &= [(A \oplus B) \ominus B]^c = \text{LHS} \\ &= (A \oplus B)^c \oplus \widehat{B} \\ &= (A^c \ominus \widehat{B}) \oplus \widehat{B} \\ &= A^c \circ \widehat{B} \end{aligned}$$